You said:

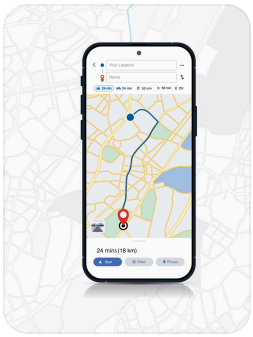
in which real life cases will dijkstra be best

ChatGPT said:

Dijkstra's algorithm is best in real-life situations where you need to find the **cheapest or shortest route from one starting point to other locations**, and every connection has a non-negative cost.

Think of it this way: you have a map, every road has a cost, and you want to find the best route without blindly checking every possible path.

5 real-life cases where Dijkstra shines



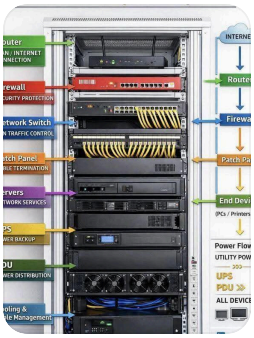
1. GPS and road navigation

Excellent use case

Find the fastest or shortest route between two places, such as your home and university.

- **Nodes:** Intersections or locations
- **Edges:** Roads
- **Weights:** Travel time or distance

Why Dijkstra? It can account for different road lengths and find the lowest-total-cost route, provided the weights are non-negative.

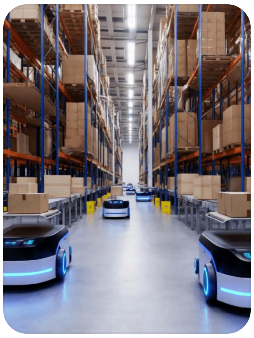


2. Computer network routing

Find an efficient path for data to travel between computers or routers.

- **Nodes:** Routers
- **Edges:** Network links
- **Weights:** Delay, link cost, or another routing metric

Why Dijkstra? It finds minimum-cost paths through a network. For example, OSPF uses Dijkstra's algorithm to calculate routes.

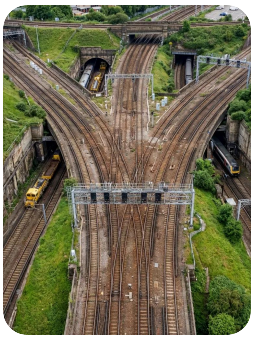


3. Robot navigation

A robot needs to move from its current position to a target while avoiding expensive or difficult routes.

- **Nodes:** Positions or grid cells
- **Edges:** Valid movements
- **Weights:** Movement cost or energy

Why Dijkstra? It can find the minimum-cost route when the environment is represented as a weighted graph. For large maps, A* may be faster when a useful heuristic is available.

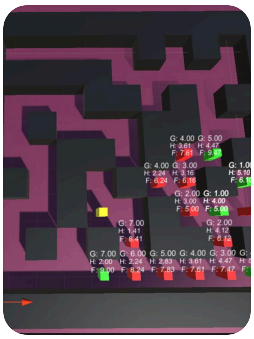


4. Railway and public transportation planning

Find the lowest-cost connection between stations.

- **Nodes:** Stations
- **Edges:** Train or transit connections
- **Weights:** Travel time, fare, or a combined cost

Why Dijkstra? It can determine the least-cost route across many connected stations, including routes with multiple transfers.



5. Video game pathfinding

An NPC needs to reach a player or destination through a game map.

- **Nodes:** Tiles or locations
- **Edges:** Possible movements
- **Weights:** Movement cost, terrain difficulty, or danger

Why Dijkstra? It finds the cheapest route even when some terrain is more expensive to cross than others.

When is Dijkstra actually the best choice?

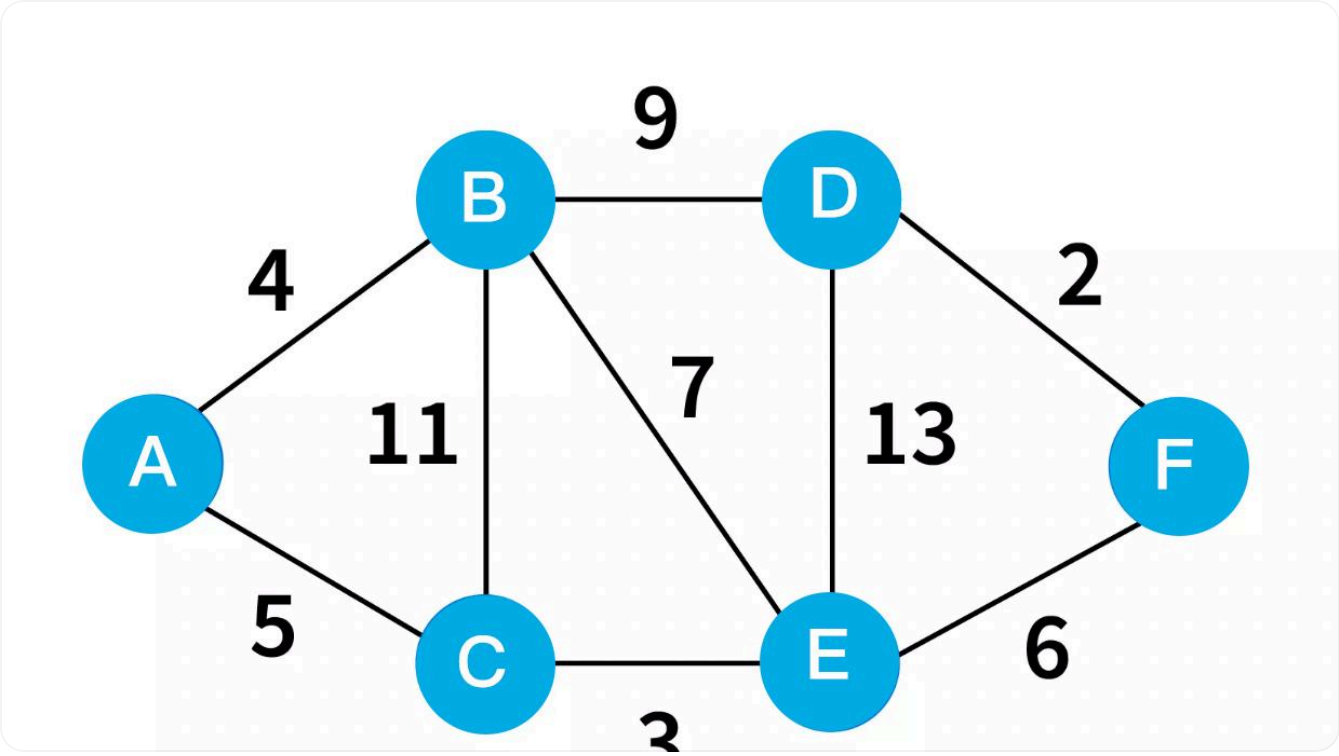
The real advantage becomes clear when you compare it with other algorithms.

Situation	Best choice	Why
Shortest path with equal edge costs	BFS	Simpler and often faster

Weighted graph with non-negative costs	Dijkstra	Reliable minimum-cost paths
One destination on a large map, with a good heuristic	A*	Can explore fewer nodes
Graph with negative edge weights	Bellman–Ford	Handles negative weights
Shortest paths between every pair of vertices	Floyd–Warshall	Solves all-pairs shortest paths
Minimum-cost spanning network	Prim or Kruskal	Connects all vertices cheaply, rather than finding routes

Let's use your graph as an example

Suppose these are travel costs between locations:



Imagine the edge labels represent travel time in minutes.

Starting at A, you want to find the shortest routes to every other location.

Rather than checking every possible route, Dijkstra repeatedly selects the unvisited location with the smallest known travel cost and updates its neighbors.

For the graph you were working with earlier:

- A → B costs 6
- A → C costs 2
- C → D costs 1
- D → B costs 5

Dijkstra discovers that reaching D through C costs only $2 + 1 = 3$, which is cheaper than any more expensive alternative. It then uses those updated distances to continue finding the cheapest routes.

One important distinction: If you only need the shortest route from A to one particular destination, Dijkstra can work very well. If you need routes from A to every location, that is also a natural use case. If you have a very large map and need just one destination, A* might be a better choice.

My recommendation for learning: implement Dijkstra in C++ using an adjacency list and a min-priority queue. That is where you'll see why it is more powerful than simply adding up every possible path.

You said:

> but in gps and road navigations, edges can be the plcae names or bus stops, how will be weights work? cant just randomly guess the weights, even if we compare weight with traffic, how does the program calculate traffic/weight, plus it needs to be dynamic too

ChatGPT said:

Exactly! You've just found one of the biggest differences between learning Dijkstra in C++ and building an actual GPS navigation system.

Dijkstra can find the best route, but it cannot tell you what the roads are like in the real world. It needs another system to provide the weights.

Let's break down how a real navigation system could work.

1. First, what exactly are the nodes, edges, and weights?

Imagine you're building a navigation system for Dhaka.

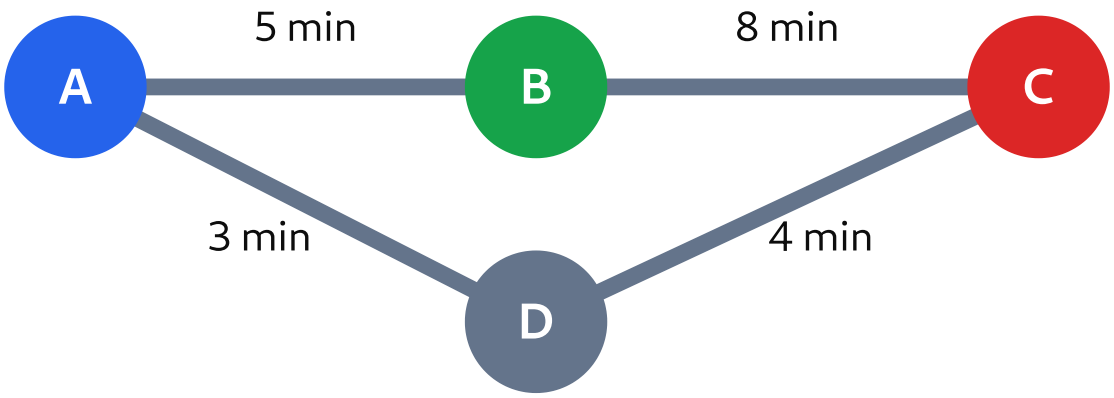


Suppose you want to travel from Mirpur to Farmgate.

- **Nodes:** Locations such as road intersections, bus stops, or station entrances.
- **Edges:** Connections between those locations, such as a road segment or a bus route.
- **Weights:** The cost of traveling along each connection.

The important detail is that a place name is not necessarily an edge. Usually, a place or intersection is a **node**, while the road connecting two places is an **edge**.

For example:



Illustrative road network. The weights are estimated travel times, not distances or real-time measurements.

If the weight of A → B is 5 minutes, that means the system estimates it will take 5 minutes to travel that road segment.

But how did the system obtain those 5 minutes?

That's the real question.

2. How does a GPS system calculate edge weights?

A navigation system can estimate travel time using several sources.

Method 1: Road distance and speed limits

Suppose a road is 4 km long and the assumed average speed is 40 km/h.

$$t = \frac{\text{distance}}{\text{speed}}$$

$$t = \frac{4}{40} \times 60 = 6 \text{ minutes}$$

The initial edge weight is 6 minutes.

This is a basic estimate. The speed limit alone doesn't guarantee that vehicles will travel at that speed.

Method 2: GPS data from real vehicles

Imagine thousands of phones are using navigation apps. With appropriate permissions and privacy safeguards, a service can analyze location and timestamp data to estimate how quickly vehicles are moving along road segments.

If vehicles usually take 12 minutes to cross a road that normally takes 6 minutes, the system may estimate a 12-minute travel time during those conditions.

This is one way traffic estimates become data-driven instead of guessed.

Method 3: Historical traffic patterns

Suppose a road usually takes:

- 6 minutes at 10 AM
- 15 minutes at 6 PM
- 8 minutes at 11 PM

Even without live traffic data, the system can predict different travel times based on the day and time.

Historical data is useful when current measurements are limited.

Method 4: Accidents, closures, and road conditions

A traffic authority or other trusted data source may report a crash, road closure, or construction.

The system can then increase the estimated travel time, mark the road as temporarily unavailable, or restrict which vehicles may use it.

Real navigation systems often combine several of these sources. The exact methods and models vary by provider.

3. But traffic changes every minute. How does Dijkstra handle that?

This is where things become interesting.

Suppose your road network initially has these estimated weights:

At 8:00 AM

Route A → B → D	Route A → C → D
12 min	**16 min**

Dijkstra chooses A → B → D because 12 minutes is less than 16 minutes.

Now imagine an accident happens on B → D, raising its estimated travel time from 7 minutes to 20 minutes.

After the traffic update

Route A → B → D	Route A → C → D
25 min	**16 min**

The system updates the edge weight and recalculates the route. The alternative is now faster.

A practical navigation system could do this as follows:

1. Receive a traffic update.
2. Update the travel-time estimate for the affected edge.
3. Determine whether the current route is still good.
4. Recalculate if necessary.
5. Notify the driver if a better route is available.

The key point: Dijkstra doesn't automatically monitor traffic. A separate data pipeline updates the weights, and the routing system responds to those updates.

For a simple implementation, rerunning Dijkstra after a meaningful traffic update is often sufficient. For a large, heavily used navigation system, continuously recalculating every route from scratch can be expensive, so more advanced techniques and optimizations may be used.

4. Let's simulate dynamic traffic

Here is a small interactive example. Change the traffic conditions and see how the best route changes.

Traffic on the B → D road

- ○ Normal traffic — 7 minutes on B → D

- Heavy traffic — 20 minutes on B → D
- Road closed — B → D is unavailable

Calculated route costs

Route 1: A → B → D — Best route

12 minutes

Route 2: A → C → D

16 minutes

Route 3: A → E → D

20 minutes

All values are illustrative. The example assumes the other segments retain the same estimated costs.

This is the basic principle behind dynamic routing: the graph's structure may remain the same while the edge weights change.

5. How would you actually build this in C++?

You don't need to build Google Maps to understand the complete concept. You can build a simplified routing engine in stages.

Stage 1 — Static weights

```
struct Edge {
    int destination;
    double travelTime;
};

vector<vector<Edge>> graph;
```

Each edge stores the destination and estimated travel time. Initially, you enter the values yourself.

Stage 2 — Calculate weights from distance and speed

```
double travelTime(double distanceKm, double speedKmh) {
    return (distanceKm / speedKmh) * 60.0;
}
```

Now weights are calculated rather than randomly guessed.

Stage 3 — Update traffic

When your program receives a new traffic estimate, update the relevant edge's travel time.

For example, an edge that previously took 5 minutes might now take 14 minutes.

Stage 4 — Recalculate the shortest route

Run Dijkstra using the updated weights.

Stage 5 — Connect real data

Instead of entering traffic estimates yourself, obtain road information from a mapping or traffic-data service. You'll need a source that provides usable road geometry, connectivity, and, where available, current or predicted travel times.

A road-map dataset alone does not necessarily contain live traffic information.

One more important detail: buses are different from roads

If you build a bus route planner, you might use bus stops as nodes and bus connections as edges. But the weight could represent more than driving time:

$$\text{Total cost} = \text{waiting time} + \text{travel time} + \text{transfer time}$$

If waiting times and departure schedules matter, the problem becomes more complex. A connection that exists at 8:00 AM may not be available at 8:10 AM. You may need a **time-dependent shortest-path algorithm** or a transit-specific routing algorithm rather than ordinary Dijkstra with fixed edge weights.

That's the distinction worth remembering:

- **Dijkstra:** Finds the minimum-cost route from the weights it is given.
- **Traffic estimation:** Determines the likely travel cost of each road.
- **Dynamic routing:** Updates those costs and decides when to find a new route.

Together, these components can form a real navigation system.